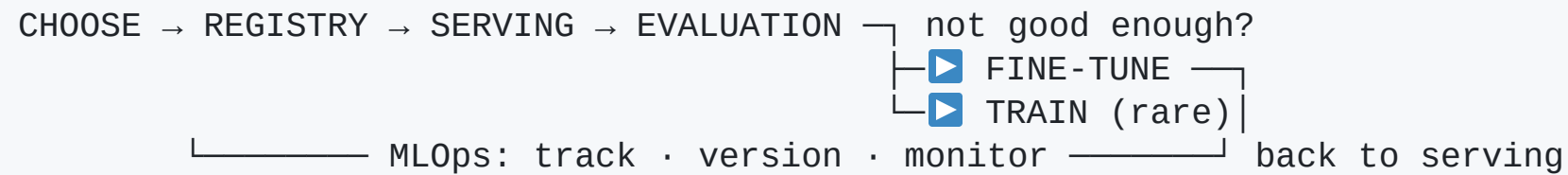


Models — deep dive

The model lifecycle

A model isn't a file you download once — it's a loop:



Two rules we keep returning to:

- Need new **knowledge**? → **RAG** (layer 3), not the model.
- Need new **behavior**? → **fine-tune**. Train from scratch almost never.

Choosing — three dimensions

- **Open vs proprietary** — runs on your hardware vs vendor API; control & privacy vs frontier ceiling
- **Size (LLM vs SLM)** — bigger = smarter but heavier; smaller = cheaper + often specialized
- **Specialization** — reasoning · tool-calling · code · language · multimodal

"Open-weights" ≠ "open-source" — you get weights, not always data or a free licence. **Read the licence.**

The one number: VRAM

$\text{VRAM}(\text{weights}) \approx \text{params} \times \text{bytes} \quad (\text{fp16}=2 \cdot \text{fp8}=1 \cdot \text{int4} \approx 0.5)$

On our **32 GB** RTX 5090:

size	fp16	int4 (Q4)	fp16 fits?	Q4 fits?
7–8B	~15 GB	~5 GB	✓	✓
14B	~28 GB	~8 GB	⚠ tight	✓
32B	~64 GB	~18 GB	✗	✓

This table *is* why quantization exists — and why a 32 GB card runs a 32B model.

Aside · superintelligence

The far end of the axis: one system above humans in *every* domain.

(*Nick Bostrom, Superintelligence, 2014*)

- **Orthogonality** — intelligence and goals are independent. *Smarter ≠ nicer.*
- **Instrumental convergence** — almost any goal → self-preserve + grab resources.

The **paperclip maximizer**: superb at the *wrong* objective.

Today's models are *narrow* — hold this as an ethical **horizon**, not a forecast. It's why alignment + human-in-the-loop matter in miniature *now*.

If we built one — four forms of agency

Bostrom Ch. 10 — least → most autonomy. Maps onto how we ship LLMs today.

Form	Core risk	≈ today
Oracle — answers only	the answer itself manipulates	Q&A chatbot
Genie — one task, then stops	literal ≠ intended ("be careful what you wish for")	task agent
Sovereign — open-ended, 24/7	no off-switch if misaligned	autonomous agent
Tool — no goals, you drive	optimization can still go agentic	function / tool call

Design heuristic for **L4 orchestration**: grant the *least* autonomy that does the job, and keep a human in the approval loop.

Serving — two engines, measure the difference

	Ollama / llama.cpp	vLLM
Best for	one user, demos	many users, throughput
Trick	GGUF + Q4 built in	PagedAttention + batching
Feel	snappy single answer	wins at concurrency

Latency = one fast answer · **throughput** = total tokens/s for everyone.

Lab 1: one model (Qwen2.5-3B) at **fp16 vs Q4** on Ollama — Q4 $\approx \frac{1}{2}$ VRAM, $\sim 1.5\times$ speed. (vLLM optional, for the batching contrast.)

Fine-tuning — teach behavior, cheaply

Full fine-tune of 7B = weights + grads + optimizer $\approx$ tens of GB $\rightarrow$ not on one card.

- **LoRA** — freeze the base, train tiny adapters ($<1\%$ of params)
- **QLoRA** — load the base in **4-bit**, train the adapter on top

method	base	VRAM (7B)
LoRA	bf16	~18–24 GB
QLoRA	4-bit	~8–12 GB

You're not moving 7B weights — you learn a few million adapter numbers.

Lab 2: turn Qwen2.5-3B into a strict-JSON order parser; track in **MLflow**.

Evaluation — numbers, not vibes

kind	what
Benchmark	MMLU / GSM8K / ARC — general ability
Task eval	<i>your</i> held-out set + a metric you chose
LLM-judge / human	preference, helpfulness

Pitfalls: **contamination** (memorized benchmarks), Q4 quality (measure it), and "one number lies" — report quality + speed + cost together.

Lab 3: base vs fine-tuned on JSON-valid % / exact-match / field accuracy.

Training & MLOps — the bookends

- **Training from scratch:** trillions of tokens + a cluster (that's why Layer 1 exists). Millions of dollars, months. *We explain*, we don't run it.
- **MLOps** is the glue: experiment tracking, model/dataset versioning, eval gates in CI, drift monitoring.

Decision tree, almost always: **use** → **fine-tune** → **(only then) train**.

Models — takeaways

1. It's a **lifecycle**, not a download: choose → serve → eval → adapt → operate
2. **VRAM** governs what runs; **quantization** is the lever
3. **Fine-tune for behavior, RAG for knowledge** — train almost never
4. **Measure** everything — quality *and* speed *and* cost
5. **MLOps** makes it reproducible